¿Qué es este proyecto?

Es una **aplicación para gestionar una colección de videojuegos retro**. Imagina una biblioteca, pero en lugar de libros hay videojuegos. La aplicación permite:

- Añadir juegos a la colección
- Ver todos los juegos organizados
- Editar o borrar juegos
- Prestar juegos a otras personas y registrar quién los tiene
- Ver estadísticas y gráficas de la colección
- Hacer todo esto tanto desde una **pantalla de texto en el terminal** como desde una **página web en el navegador**

¿Qué tecnologías se han usado?

Todo está escrito en **Python**, que es un lenguaje de programación muy popular. Dentro de Python se han usado varias **librerías** — piensa en ellas como herramientas especializadas que alguien ya ha creado y que puedes usar gratis.

Las librerías usadas son:

Django — es el framework con el que se construye la parte web. Un framework es como un kit de construcción: te da la estructura básica para hacer una web sin tener que inventarlo todo desde cero. Se instala con:

```
pip install django
```

SQLite — es la base de datos donde se guardan los juegos. Una base de datos es como una hoja de cálculo muy organizada y segura. SQLite viene incluida con Python, no hay que instalarla.

Pandas — librería para analizar datos. Permite cargar los juegos de la base de datos y hacer cálculos sobre ellos (cuántos hay por plataforma, cómo ha crecido la colección...). Se instala con:

```
pip install pandas
```

Matplotlib — librería para hacer gráficas. Con ella se generan los tres gráficos del módulo de análisis. Se instala con:

```
pip install matplotlib
```

Pytest — librería para hacer pruebas automáticas del código. Permite comprobar que todo funciona correctamente de forma automática. Se instala con:

```
pip install pytest
```

¿Qué archivos hay y para qué sirve cada uno?

```
P1/
├── models.py          ← Las clases que representan los videojuegos
├── database.py        ← Todo lo relacionado con guardar y leer datos
├── interface.py       ← La interfaz de texto (consola)
├── analysis.py        ← Las gráficas y estadísticas
├── main.py            ← El menú principal de la consola
├── generar_datos.py   ← Script para poblar la BD con datos de ejemplo
├── manage.py          ← Comando principal de Django (lo genera Django)
├── requirements.txt   ← Lista de librerías necesarias
├── videoteca.db       ← La base de datos (se crea automáticamente)
│
├── videoteca/         ← Configuración global de Django
│   ├── settings.py    ← Ajustes: BD, idioma, apps instaladas...
│   └── urls.py        ← Rutas globales de la web
│
├── catalogo/          ← La aplicación web
│   ├── views.py       ← Lógica de cada página web
│   ├── urls.py        ← Rutas de la aplicación
│   └── templates/catalogo/
│       ├── base.html              ← Plantilla madre (navegación, estilos)
│       ├── coleccion.html         ← Página principal con la tabla
│       ├── formulario_juego.html  ← Formulario de añadir/editar
│       ├── confirmar_borrado.html ← Pantalla de confirmación al borrar
│       ├── formulario_prestamo.html ← Formulario de préstamo
│       └── analisis.html         ← Página con las gráficas
│
└── tests/
    ├── test_models.py    ← Pruebas de las clases
    ├── test_database.py  ← Pruebas de la base de datos
    ├── test_analysis.py  ← Pruebas del análisis
    └── test_robustez.py  ← Pruebas de entradas incorrectas
```

El archivo `models.py` — Las clases que representan los videojuegos

¿Qué es una clase?

Antes de entrar en el código, necesitas entender qué es una **clase** en programación.

Imagina que quieres describir un coche. Un coche tiene características (color, marca, año) y cosas que puede hacer (arrancar, frenar, acelerar). En programación, una **clase** es exactamente eso: una plantilla que describe cómo es un objeto y qué puede hacer.

En este proyecto, los objetos son videojuegos. Cada videojuego tiene características (título, plataforma, año, género) y cosas que puede hacer (mostrar sus detalles).

La clase `Item` — la plantilla base

```python
from abc import ABC, abstractmethod
from datetime import datetime

class Item(ABC):
```

`Item` es la clase base. Es **abstracta** — esto significa que es solo una plantilla, no se puede crear un objeto directamente de ella. Es como decir "esto es un vehículo" sin especificar si es un coche, una moto o un camión.

`ABC` viene de la librería `abc` de Python y es lo que hace que la clase sea abstracta.

Los atributos privados

```python
def __init__(self, titulo, plataforma, anio, genero):
    self.__titulo = titulo
    self.__plataforma = plataforma
    self.__anio = self._validar_anio(anio)
    self.__genero = genero
```

`__init__` es el **constructor** — es el método que se ejecuta automáticamente cuando se crea un objeto. Aquí se guardan las características del videojuego.

El doble guion bajo `__` delante de cada nombre (como `__titulo`) hace que el atributo sea **privado**. Esto significa que nadie puede modificarlo directamente desde fuera de la clase. Es como una caja fuerte: los datos están protegidos.

Fíjate en `self._validar_anio(anio)` — antes de guardar el año, lo valida llamando a otro método.

La validación del año

```python
def _validar_anio(self, anio):
    if anio > datetime.now().year:
        raise ValueError("El año no puede ser superior al actual.")
    return anio
```

Este método comprueba que el año del juego no sea futuro. `datetime.now().year` obtiene el año actual del sistema. Si alguien intenta añadir un juego con año 2099, Python lanzará un error (`ValueError`) con un mensaje explicativo.

Las @property — acceso controlado a los datos

```python
@property
def titulo(self):
    return self.__titulo
```

Como los atributos son privados, necesitamos una forma de leerlos desde fuera. Los `@property` son métodos que funcionan como si fueran atributos normales. Desde fuera de la clase puedes escribir `juego.titulo` y obtienes el título, pero no puedes escribir `juego.titulo = "otro"` para cambiarlo sin permiso.

El método abstracto

```python
@abstractmethod
def mostrar_detalle(self) -> str:
    pass
```

Este método está declarado pero **no tiene código** — solo dice "cualquier clase que herede de Item DEBE implementar este método". Es un contrato obligatorio.

Las subclases — JuegoFisico y JuegoDigital

Aquí es donde entra la **herencia**. Herencia significa que una clase puede basarse en otra y heredar todo lo que tiene, añadiendo sus propias cosas.

```python
class JuegoFisico(Item):
    def __init__(self, titulo, plataforma, anio, genero, estanteria):
        super().__init__(titulo, plataforma, anio, genero)
        self.estanteria = estanteria

    def mostrar_detalle(self) -> str:
        return f"[FÍSICO] {self.titulo} - Estante: {self.estanteria}"
```

`JuegoFisico` hereda de `Item` — esto significa que tiene todo lo que tiene `Item` (título, plataforma, año, género) y además añade `estanteria`.

`super().__init__(...)` llama al constructor de la clase padre (`Item`) para que inicialice los atributos comunes.

`mostrar_detalle()` aquí sí tiene código — devuelve una cadena de texto con los detalles del juego físico.

```python
class JuegoDigital(Item):
    def __init__(self, titulo, plataforma, anio, genero, enlace):
        super().__init__(titulo, plataforma, anio, genero)
        self.enlace = enlace

    def mostrar_detalle(self) -> str:
        return f"[DIGITAL] {self.titulo} - Enlace: {self.enlace}"
```

`JuegoDigital` es igual pero en lugar de estantería tiene un enlace de descarga.

¿Por qué esta estructura?

La ventaja de este diseño es el **polimorfismo**. Imagina que tienes una lista con juegos físicos y digitales mezclados. Puedes llamar a `mostrar_detalle()` en cualquiera de ellos sin saber de qué tipo es — cada uno responderá de forma diferente pero correcta. Es como decirle a cualquier vehículo "arranca" — cada uno lo hará a su manera.

El archivo `database.py` — Guardar y leer datos

¿Qué es una base de datos?

Imagina una hoja de cálculo Excel con dos pestañas: una llamada `juegos` y otra llamada `prestamos`. Cada fila es un juego o un préstamo, y cada columna es una característica (título, plataforma, año...).

Una base de datos es exactamente eso, pero mucho más organizada, segura y potente. En este proyecto se usa **SQLite**, que guarda toda la información en un único archivo llamado `videoteca.db`.

La clase `DatabaseManager`

```python
class DatabaseManager:
    def __init__(self, db_name="videoteca.db"):
        self.conn = sqlite3.connect(db_name)
        self._crear_tablas()
```

Esta clase es la **única puerta de entrada** a la base de datos. Todo el proyecto pasa por aquí para leer o guardar datos — ningún otro archivo habla con la base de datos directamente.

`sqlite3.connect(db_name)` abre la conexión con el archivo de base de datos. Si el archivo no existe, lo crea automáticamente.

Creación de las tablas

```python
def _crear_tablas(self):
    with self.conn:
        self.conn.execute("""
            CREATE TABLE IF NOT EXISTS juegos (
                id         INTEGER PRIMARY KEY AUTOINCREMENT,
                titulo     TEXT NOT NULL,
                plataforma TEXT NOT NULL,
                anio       INTEGER NOT NULL,
                genero     TEXT NOT NULL,
                tipo       TEXT NOT NULL,
                extra      TEXT
            )
        """)
        self.conn.execute("""
            CREATE TABLE IF NOT EXISTS prestamos (
                juego_id     INTEGER PRIMARY KEY,
                destinatario TEXT NOT NULL,
                fecha_inicio TEXT,
                FOREIGN KEY (juego_id) REFERENCES juegos(id)
            )
        """)
```

Esto crea las dos tablas si no existen todavía. `IF NOT EXISTS` significa "créala solo si aún no existe" — así el código no falla si la base de datos ya tiene datos.

La tabla `juegos` tiene estas columnas:

- `id` — número único que identifica cada juego, se asigna automáticamente
- `titulo`, `plataforma`, `anio`, `genero`, `tipo` — datos del juego, obligatorios (`NOT NULL`)
- `extra` — estantería o URL, opcional

La tabla `prestamos` es especial: `juego_id` es a la vez **clave primaria** y **clave foránea**. Esto significa dos cosas:

- Como clave primaria: no puede haber dos filas con el mismo `juego_id` — un juego solo puede estar prestado a una persona a la vez, garantizado por la propia base de datos
- Como clave foránea: hace referencia a la tabla `juegos` — no puedes prestar un juego que no existe

---

Insertar un juego

```python
def insertar_juego(self, titulo, plataforma, anio, genero, tipo, extra):
    query = """
        INSERT INTO juegos (titulo, plataforma, anio, genero, tipo, extra)
        VALUES (?, ?, ?, ?, ?, ?)
    """
    with self.conn:
        self.conn.execute(query, (titulo, plataforma, anio, genero, tipo, extra))
```

`INSERT INTO` es el comando SQL para añadir una fila nueva a la tabla.

Los signos de interrogación `?` son muy importantes — son **marcadores de posición**. En lugar de escribir los valores directamente en la consulta (lo que sería peligroso), se pasan por separado. Esto evita un ataque muy común llamado **inyección SQL**, donde alguien podría escribir código malicioso en un campo de texto para manipular la base de datos.

`with self.conn:` garantiza que si algo falla a mitad de la operación, los cambios se deshacen automáticamente — o todo se guarda o nada se guarda.

Leer todos los juegos

```python
def obtener_coleccion_completa(self):
    query = """
        SELECT j.id, j.titulo, j.plataforma, j.anio, j.genero,
               j.tipo, j.extra, p.destinatario
        FROM juegos j
        LEFT JOIN prestamos p ON j.id = p.juego_id
        ORDER BY j.plataforma, j.titulo
    """
    return self.conn.execute(query).fetchall()
```

Esta es la consulta más importante del proyecto. `SELECT` recupera datos de la base de datos.

`LEFT JOIN` combina las dos tablas — juegos y préstamos — en una sola consulta. Funciona así: para cada juego, busca si hay un préstamo activo. Si lo hay, añade el nombre del destinatario. Si no lo hay, pone `NULL` (vacío). Así con una sola consulta obtenemos toda la información necesaria para mostrar la colección completa.

`ORDER BY j.plataforma, j.titulo` ordena los resultados primero por plataforma y luego por título dentro de cada plataforma.

Editar y eliminar

```python
def editar_juego(self, id_juego, titulo, plataforma, anio, genero, extra):
    query = """
        UPDATE juegos
        SET titulo=?, plataforma=?, anio=?, genero=?, extra=?
        WHERE id=?
    """
    with self.conn:
        self.conn.execute(query, (titulo, plataforma, anio, genero, extra, id_juego)
```

`UPDATE` modifica una fila existente. `WHERE id=?` es crucial — sin él se modificarían TODOS los juegos de la tabla.

```python
def eliminar_juego(self, id_juego):
    with self.conn:
        self.conn.execute("DELETE FROM prestamos WHERE juego_id = ?", (id_juego,))
        self.conn.execute("DELETE FROM juegos WHERE id = ?", (id_juego,))
```

Fíjate que primero se borra el préstamo y luego el juego — si fuera al revés, la clave foránea lo impediría porque el préstamo hace referencia al juego.

Gestión de préstamos

```python
def registrar_prestamo(self, id_juego, destinatario):
    fecha = datetime.now().strftime("%Y-%m-%d")
    try:
        with self.conn:
            self.conn.execute(
                "INSERT INTO prestamos VALUES (?, ?, ?)",
                (id_juego, destinatario, fecha)
            )
    except sqlite3.IntegrityError:
        print("\nError: El videojuego ya está marcado como prestado.")
```

`datetime.now().strftime("%Y-%m-%d")` obtiene la fecha actual en formato año-mes-día.

El bloque `try/except` captura el error que lanzaría SQLite si intentáramos prestar un juego que ya está prestado — recordemos que `juego_id` es clave primaria y no puede repetirse. En lugar de que el programa se rompa, muestra un mensaje de error amigable.

Los archivos `interface.py` y `main.py` — La interfaz de consola

¿Qué es la interfaz de consola?

Es la versión del programa que funciona en el terminal — la pantalla de texto negra donde escribes comandos. No tiene botones ni colores bonitos, solo texto. Es la forma más básica de interactuar con el programa.

El archivo `interface.py` — Pedir datos al usuario

Esta clase se encarga de todo lo relacionado con mostrar información en pantalla y pedir datos al usuario. Su trabajo es **separar la presentación de la lógica** — `interface.py` no sabe nada de base de datos, solo sabe mostrar cosas y pedir cosas.

El menú principal

```python
def mostrar_menu(self):
    print("\n" + "=" * 55)
    print("        SISTEMA DE GESTIÓN DE VIDEOTECA RETRO")
    print("=" * 55)
    print("1. Añadir Juego Físico    | 2. Añadir Juego Digital")
    print("3. Ver Colección Completa | 4. EDITAR Videojuego")
    print("5. ELIMINAR Videojuego    | 6. Prestar")
    print("7. Devolver               | 8. Análisis Estadístico")
    print("9. Salir")
    return input("\nSeleccione una opción: ").strip()
```

`print()` muestra texto en pantalla. `"=" * 55` repite el carácter `=` 55 veces para hacer una línea decorativa.

`input()` espera a que el usuario escriba algo y pulse Enter, y devuelve lo que escribió como texto.

`.strip()` elimina los espacios en blanco que el usuario pudiera haber escrito por error al principio o al final.

La validación de entradas

Esta es una de las partes más importantes de `interface.py`. Imagina que el programa te pide el año de lanzamiento y escribes "hola" — sin validación el programa se rompería. Con validación, te pide el dato de nuevo hasta que escribas algo correcto.

Validar texto

```python
def _pedir_texto(self, campo):
    while True:
        valor = input(f"{campo}: ").strip()
        if valor:
            return valor
        print(f"  Error: el campo '{campo}' no puede estar vacío.")
```

`while True:` crea un bucle infinito — el código dentro se repite forever hasta que algo lo pare.

`if valor:` comprueba que el texto no esté vacío. Si el usuario pulsa Enter sin escribir nada, `valor` estará vacío y la condición será falsa, por lo que mostrará el error y volverá a pedir el dato.

`return valor` para el bucle y devuelve el valor cuando es válido.

Validar el año

```python
def _pedir_anio(self):
    anio_actual = datetime.now().year
    while True:
        entrada = input("Año de lanzamiento: ").strip()
        if not entrada.isdigit():
            print("  Error: introduce un número entero (ej. 1993).")
            continue
        anio = int(entrada)
        if anio < 1970:
            print("  Error: el año no puede ser anterior a 1970.")
        elif anio > anio_actual:
            print(f"  Error: el año no puede ser superior a {anio_actual}.")
        else:
            return anio
```

Esta validación tiene tres niveles:

1. `.isdigit()` comprueba que lo escrito sean solo números — si alguien escribe "mil novecientos" o "abc" lo rechaza
2. Comprueba que el año no sea anterior a 1970 — no tiene sentido en el contexto retro
3. Comprueba que no sea futuro — coherente con la validación del modelo

`continue` salta al inicio del bucle sin ejecutar el resto del código.

Validar un ID

```python
def pedir_id(self, accion):
    while True:
        entrada = input(f"ID del videojuego a {accion}: ").strip()
        if entrada.isdigit() and int(entrada) > 0:
            return int(entrada)
        print("  Error: introduce un número entero positivo.")
```

Simple pero importante — garantiza que el ID sea un número entero positivo antes de enviarlo a la base de datos.

Mostrar la colección en tabla

```python
def mostrar_coleccion_tabular(self, registros):
    if not registros:
        print("\nLa videoteca está vacía.")
        return

    plataforma_actual = None

    for registro in registros:
        idx, titulo, plataforma, anio, genero, tipo, extra, dest = registro

        if plataforma != plataforma_actual:
            plataforma_actual = plataforma
            print(f"\n--- PLATAFORMA: {plataforma_actual.upper()} ---")

        estado = f"Prestado a: {dest}" if dest else "En Biblioteca"
        print(f"{idx:<4} | {titulo:<25} | {anio:<6} | {genero:<15} | {estado}")
```

`for registro in registros:` recorre todos los juegos uno a uno.

`idx, titulo, plataforma, ... = registro` desempaqueta la tupla — es una forma elegante de asignar cada valor de la fila a una variable con nombre descriptivo.

`if plataforma != plataforma_actual:` detecta cuando cambia de plataforma para imprimir el encabezado de grupo.

`f"Prestado a: {dest}" if dest else "En Biblioteca"` es un condicional en una línea — si `dest` tiene valor muestra quién lo tiene, si es `None` muestra "En Biblioteca".

`{titulo:<25}` formatea el texto para que ocupe exactamente 25 caracteres, alineado a la izquierda — así todas las columnas quedan al[illegible]das.

El archivo `main.py` — El menú principal

Este es el archivo que arranca todo cuando ejecutas `python main.py`. Es el **director de orquesta** — no hace casi nada por sí mismo, simplemente coordina las otras clases.

```python
def ejecutar_app():
    db = DatabaseManager()
    ui = InterfazConsola()
    analizador = AnalizadorColeccion()

    while True:
        opcion = ui.mostrar_menu()

        try:
            if opcion in ["1", "2"]:
                tipo = "Físico" if opcion == "1" else "Digital"
                titulo, plataforma, anio, genero = ui.pedir_datos_juego()
                extra = ui.pedir_texto_extra(tipo)
                db.insertar_juego(titulo, plataforma, anio, genero, tipo, extra)
                print("  Videojuego añadido con éxito.")

            elif opcion == "3":
                ui.mostrar_coleccion_tabular(db.obtener_coleccion_completa())

            elif opcion == "9":
                print("\nCerrando el sistema. ¡Hasta pronto!")
                break

        except Exception as e:
            print(f"\nERROR INESPERADO: {e}")
```

`db = DatabaseManager()` crea la conexión con la base de datos. `ui = InterfazConsola()` crea la interfaz de usuario. `analizador = AnalizadorColeccion()` crea el módulo de análisis.

`while True:` mantiene el programa funcionando indefinidamente hasta que el usuario elige la opción 9.

`try/except Exception` captura cualquier error inesperado — en lugar de que el programa se cierre abruptamente, muestra el error y vuelve al menú.

`break` rompe el bucle infinito y termina el programa.

¿Por qué separar en tres archivos?

Podría haberse escrito todo en un solo archivo, pero separarlo tiene ventajas importantes:

- Si quieres cambiar cómo se muestran los datos, solo tocas `interface.py`
- Si quieres cambiar la base de datos, solo tocas `database.py`
- Cada parte puede probarse de forma independiente
- El código es más fácil de entender y mantener

Esto se llama **separación de responsabilidades** y es un principio fundamental del buen diseño de software.

El archivo `analysis.py` — Gráficas y estadísticas

¿Qué hace este módulo?

Transforma los datos de la base de datos en información visual. En lugar de ver una lista de juegos, el usuario puede ver gráficas que responden preguntas como: ¿qué plataforma tiene más juegos? ¿cómo ha crecido la colección con los años? ¿qué géneros predominan en cada década?

La clase `AnalizadorColeccion`

```python
class AnalizadorColeccion:
    def __init__(self, db_name="videoteca.db"):
        self.db_name = db_name
```

Simple — solo guarda el nombre del archivo de base de datos para usarlo después.

Cargar y limpiar los datos

```python
def _cargar_datos(self):
    conn = sqlite3.connect(self.db_name)
    df = pd.read_sql_query(
        "SELECT plataforma, anio, genero FROM juegos", conn
    )
    conn.close()

    df = df.dropna(subset=["plataforma", "anio", "genero"])
    df["plataforma"] = df["plataforma"].str.strip()
    df["genero"]     = df["genero"].str.strip()
    df["anio"]       = df["anio"].astype(int)

    return df
```

`pd.read_sql_query()` es de Pandas — ejecuta una consulta SQL y devuelve el resultado como un **DataFrame**. Un DataFrame es como una tabla Excel en memoria: tiene filas, columnas y se puede manipular fácilmente.

Después se aplica un **pipeline de limpieza** en tres pasos:

`df.dropna()` elimina las filas que tengan valores vacíos en las columnas importantes. Si un juego no tiene plataforma o género registrado, no sirve para el análisis.

`str.strip()` elimina espacios extra en los textos. Esto evita que `"NES"` y `"NES "` (con un espacio al final) se traten como plataformas distintas y aparezcan como dos barras separadas en la gráfica.

`astype(int)` convierte el año a número entero. SQLite a veces devuelve los números como decimales (1986.0 en lugar de 1986) y esto lo corrige.

Generar las tres gráficas

```python
def ejecutar_analisis_completo(self):
    df = self._cargar_datos()

    if df.empty:
        print("\nNo hay datos suficientes para el análisis.")
        return

    fig, (ax1, ax2, ax3) = plt.subplots(1, 3, figsize=(18, 5))
    fig.suptitle("Análisis de la Colección de Videojuegos Retro", fontsize=14)
```

`plt.subplots(1, 3)` crea una figura con 3 gráficas en una fila. `fig` es la figura completa y `ax1, ax2, ax3` son cada una de las tres gráficas individuales.

`figsize=(18, 5)` define el tamaño en pulgadas — 18 de ancho y 5 de alto.

Gráfica 1 — Distribución por plataforma

```python
conteo = df["plataforma"].value_counts()
conteo.plot(kind="bar", ax=ax1, color="skyblue")
ax1.set_title("Juegos por plataforma")
ax1.set_xlabel("Plataforma")
ax1.set_ylabel("Número de juegos")
ax1.tick_params(axis="x", rotation=45)
```

`df["plataforma"].value_counts()` cuenta cuántos juegos hay de cada plataforma y los ordena de mayor a menor. El resultado sería algo como: `NES: 3, PC: 2, PS4: 2...`

`.plot(kind="bar")` convierte ese conteo en un gráfico de barras.

`rotation=45` rota las etiquetas del eje X 45 grados para que no se solapen si los nombres son largos.

Gráfica 2 — Evolución temporal

```python
temporal = df.groupby("anio").size().cumsum()
temporal.plot(kind="line", ax=ax2, marker="o", color="green")
ax2.set_title("Evolución de la colección")
```

`df.groupby("anio").size()` agrupa los juegos por año y cuenta cuántos hay en cada año. El resultado sería algo como: `1986: 2, 1991: 1, 1993: 1...`

`.cumsum()` calcula la **suma acumulada** — en lugar de ver cuántos juegos se añadieron cada año, ves el total acumulado. Así la gráfica siempre sube y muestra el crecimiento total de la colección.

`marker="o"` añade un punto en cada año para que sea más fácil de leer.

Gráfica 3 — Correlación género/año

```python
ax3.scatter(df["anio"], df["genero"], alpha=0.6, color="purple")
ax3.set_title("Correlación género / lanzamiento")
ax3.set_xlabel("Año")
```

`scatter()` crea un diagrama de dispersión — cada punto representa un juego, con el año en el eje horizontal y el género en el eje vertical. Permite ver visualmente si ciertos géneros predominan en ciertas décadas.

`alpha=0.6` hace los puntos semitransparentes — si hay varios juegos del mismo género y año, los puntos se superponen pero se pueden distinguir por la transparencia.

Mostrar las gráficas

```python
plt.tight_layout()
plt.show()
```

`tight_layout()` ajusta automáticamente los márgenes para que las gráficas no se solapen entre sí.

`plt.show()` abre la ventana con las gráficas. El programa se pausa hasta que el usuario cierra esa ventana.

¿Cómo funciona en la web?

En la interfaz web las gráficas no pueden abrirse en una ventana separada — tienen que incrustarse en la página HTML. Para eso se usa un truco:

```python
buffer = io.BytesIO()
plt.savefig(buffer, format='png', bbox_inches='tight')
buffer.seek(0)
grafica_b64 = base64.b64encode(buffer.read()).decode('utf-8')
plt.close(fig)
```

En lugar de mostrar la gráfica en pantalla, se guarda en memoria como si fuera un archivo PNG. Luego ese archivo se convierte a texto usando **Base64** — un sistema que convierte cualquier archivo binario (imagen, audio, vídeo) en una cadena de texto. Esa cadena de texto se incrusta directamente en el HTML:

```html
<img src="{{ grafica }}">
```

El navegador recibe ese texto, lo reconoce como una imagen y la muestra. Todo sin escribir ningún archivo en disco.

La interfaz web — Django

¿Qué es Django y cómo funciona?

Django es un framework web para Python. Un framework es como un kit de construcción — te da la estructura básica para hacer una web sin tener que inventarlo todo desde cero.

Django sigue un patrón llamado **MTV (Model-Template-View)**. Es similar al famoso MVC que quizás hayas oído. Funciona así:

- **Model (Modelo)** — los datos. En este proyecto es la base de datos SQLite gestionada por `DatabaseManager`
- **Template (Plantilla)** — lo que ve el usuario. Son los archivos HTML
- **View (Vista)** — la lógica. Recibe la petición del usuario, consulta los datos y devuelve la página HTML

Cuando el usuario abre el navegador y escribe `http://127.0.0.1:8000`, esto es lo que ocurre:

```
Navegador → URLs → Vista → DatabaseManager → Plantilla HTML → Navegador
```

1. El navegador hace una petición
2. Django mira las URLs para saber qué vista debe ejecutar
3. La vista consulta los datos
4. La vista rellena la plantilla HTML con esos datos
5. Django devuelve la página al navegador

La configuración — `videoteca/settings.py`

Este archivo configura todo el proyecto Django. Los cambios más importantes que se hicieron fueron:

```python
INSTALLED_APPS = [
    ...
    'catalogo',    # ← nuestra aplicación
]

DATABASES = {
    'default': {
        'ENGINE': 'django.db.backends.sqlite3',
        'NAME': BASE_DIR / 'videoteca.db',
    }
}

LANGUAGE_CODE = 'es-es'
TIME_ZONE = 'Europe/Madrid'
```

`INSTALLED_APPS` le dice a Django qué aplicaciones forman el proyecto. Sin añadir `catalogo` aquí, Django no la reconocería.

`DATABASES` indica qué base de datos usar y dónde está el archivo. `BASE_DIR` es la carpeta raíz del proyecto.

Las URLs — cómo se enrutan las peticiones

`videoteca/urls.py` — el enrutador global

```python
from django.urls import path, include

urlpatterns = [
    path('admin/', admin.site.urls),
    path('', include('catalogo.urls')),
]
```

Este archivo es el punto de entrada de todas las peticiones. La línea `path('', include('catalogo.urls'))` dice: "para cualquier URL que no sea `/admin/`, mírala en `catalogo/urls.py`".

`catalogo/urls.py` — las rutas de la aplicación

```python
urlpatterns = [
    path('', views.coleccion, name='coleccion'),
    path('añadir/', views.añadir_juego, name='añadir_juego'),
    path('editar/<int:id_juego>/', views.editar_juego, name='editar_juego'),
    path('eliminar/<int:id_juego>/', views.eliminar_juego, name='eliminar_juego'),
    path('prestar/<int:id_juego>/', views.prestar_juego, name='prestar_juego'),
    path('devolver/<int:id_juego>/', views.devolver_juego, name='devolver_juego'),
    path('analisis/', views.analisis, name='analisis'),
]
```

Cada línea conecta una URL con una función de `views.py`. Por ejemplo:

- Cuando el usuario va a `http://127.0.0.1:8000/` → se ejecuta `views.coleccion`
- Cuando va a `http://127.0.0.1:8000/añadir/` → se ejecuta `views.añadir_juego`
- Cuando va a `http://127.0.0.1:8000/editar/3/` → se ejecuta `views.editar_juego` con `id_juego=3`

`<int:id_juego>` es un **conversor de tipo** — extrae el número de la URL y lo pasa a la vista como un entero. Si alguien escribe letras en lugar de números, Django devuelve un error 404 automáticamente.

`name='coleccion'` da un nombre a cada ruta. Esto permite referenciarlas desde las plantillas sin escribir la URL directamente — si la URL cambia en el futuro, solo hay que cambiarla en un sitio.

Las vistas — `catalogo/views.py`

Las vistas son funciones Python que reciben una petición HTTP y devuelven una respuesta. Aquí están las más importantes:

Vista de la colección

```python
def coleccion(request):
    db = _get_db()
    registros = db.obtener_coleccion_completa()
    db.conn.close()
    return render(request, 'catalogo/coleccion.html', {'registros': registros})
```

`request` contiene toda la información de la petición del navegador.

`_get_db()` crea una instancia de `DatabaseManager` — así la vista puede hablar con la base de datos.

`db.obtener_coleccion_completa()` ejecuta el LEFT JOIN que vimos antes y devuelve todos los juegos.

`render()` es la función clave de Django — toma la plantilla HTML, la rellena con los datos (`{'registros': registros}`) y devuelve la página completa al navegador.

Vista de añadir juego

```python
def añadir_juego(request):
    if request.method == 'POST':
        titulo     = request.POST.get('titulo', '').strip()
        plataforma = request.POST.get('plataforma', '').strip()
        anio       = request.POST.get('anio', '').strip()
        genero     = request.POST.get('genero', '').strip()
        tipo       = request.POST.get('tipo', 'Físico')
        extra      = request.POST.get('extra', '').strip()

        if not all([titulo, plataforma, anio, genero, extra]):
            messages.error(request, 'Todos los campos son obligatorios.')
            return render(request, 'catalogo/formulario_juego.html',
                          {'accion': 'Añadir', 'datos': request.POST})

        if not anio.isdigit():
            messages.error(request, 'El año debe ser un número entero.')
            return render(request, 'catalogo/formulario_juego.html',
                          {'accion': 'Añadir', 'datos': request.POST})

        try:
            db = _get_db()
            db.insertar_juego(titulo, plataforma, int(anio), genero, tipo, extra)
            db.conn.close()
            messages.success(request, f'"{titulo}" añadido correctamente.')
            return redirect('coleccion')
        except ValueError as e:
            messages.error(request, str(e))

    return render(request, 'catalogo/formulario_juego.html', {'accion': 'Añadir'})
```

Esta vista maneja dos situaciones distintas según el método HTTP:

GET — el usuario ha navegado al formulario por primera vez. Se muestra el formulario vacío. `request.method == 'POST'` es falso, así que se salta todo el bloque y va directamente al `return render()` del final.

POST — el usuario ha rellenado el formulario y pulsado "Añadir". Los datos llegan en `request.POST`. Se validan y si todo está bien se guardan en la base de datos.

`request.POST.get('titulo', '')` obtiene el valor del campo `titulo` del formulario. El segundo parámetro `''` es el valor por defecto si el campo no existe.

`not all([titulo, plataforma, anio, genero, extra])` comprueba que ningún campo esté vacío — `all()` devuelve `True` solo si todos los elementos son verdaderos (no vacíos).

`messages.error()` y `messages.success()` son el sistema de notificaciones de Django — guardan un mensaje en la sesión del usuario que se muestra en la siguiente página.

`return redirect('coleccion')` redirige al usuario a la página principal tras guardar — esto implementa el patrón **POST/Redirect/GET** que evita que al recargar la página se duplique la operación.

Las plantillas HTML

Las plantillas son archivos HTML con código Django especial entre llaves. Django las rellena con datos reales antes de enviarlas al navegador.

`base.html` — la plantilla madre

```html
<!DOCTYPE html>
<html lang="es">
<head>
    <title>{% block titulo %}Videoteca Retro{% endblock %}</title>
</head>
<body>
<nav>
    <a href="{% url 'coleccion' %}">🎮 Videoteca Retro</a>
    <a href="{% url 'añadir_juego' %}">+ Añadir juego</a>
    <a href="{% url 'analisis' %}">Análisis</a>
</nav>

{% if messages %}
    {% for message in messages %}
        <div class="msg {{ message.tags }}">{{ message }}</div>
    {% endfor %}
{% endif %}

{% block contenido %}{% endblock %}
</body>
</html>
```

`{% block titulo %}` y `{% block contenido %}` son **bloques** — huecos que las plantillas hijas pueden rellenar con su propio contenido.

`{% url 'coleccion' %}` genera la URL correcta a partir del nombre de la ruta — si la URL cambia en `urls.py`, todos los enlaces se actualizan automáticamente.

`{% if messages %}` comprueba si hay mensajes pendientes para mostrar al usuario.

`coleccion.html` — la plantilla hija

html

```html
{% extends 'catalogo/base.html' %}

{% block contenido %}
<h1>Mi Colección</h1>
<table>
    <thead>
        <tr>
            <th>ID</th><th>Título</th><th>Año</th>...
        </tr>
    </thead>
    <tbody>
        {% for r in registros %}
        <tr>
            <td>{{ r.0 }}</td>
            <td>{{ r.1 }}</td>
            <td>{{ r.3 }}</td>
            <td>
                {% if r.7 %}
                    📤 {{ r.7 }}
                {% else %}
                    ✅ En biblioteca
                {% endif %}
            </td>
            <td>
                <a href="{% url 'editar_juego' r.0 %}">Editar</a>
                <a href="{% url 'eliminar_juego' r.0 %}">Borrar</a>
            </td>
        </tr>
        {% endfor %}
    </tbody>
</table>
{% endblock %}
```

`{% extends 'catalogo/base.html' %}` indica que esta plantilla hereda de `base.html` — esto implementa el principio DRY (no repetir código).

`{% for r in registros %}` recorre todos los juegos que la vista ha pasado a la plantilla. Para cada juego crea una fila en la tabla.

`{{ r.0 }}`, `{{ r.1 }}` acceden a cada columna de la tupla que devuelve la base de datos — r.0 es el ID, r.1 el título, r.3 el año, r.7 el destinatario del préstamo.

`{% if r.7 %}` comprueba si hay préstamo activo — si `r.7` tiene valor muestra quién lo tiene, si es `None` muestra "En biblioteca".

La seguridad

Se implementaron cuatro medidas de seguridad:

Token CSRF — en cada formulario hay `{% csrf_token %}`. Django genera un código secreto único por sesión. Si alguien intenta enviar un formulario desde otra página web, el código no coincidirá y Django rechazará la petición con un error 403.

Validación en servidor — aunque el HTML tiene `required` en los campos, la vista siempre valida los datos en Python. Las restricciones del navegador se pueden saltar fácilmente con herramientas de desarrollo.

Consultas parametrizadas — todos los valores van con `?` en las queries SQL, nunca concatenados directamente, eliminando el riesgo de inyección SQL.

Confirmación de borrado — eliminar un juego requiere dos pasos: primero una petición GET que muestra la pantalla de confirmación, y solo una petición POST explícita ejecuta el borrado.

Los tests — Pruebas automáticas del código

¿Qué son los tests y para qué sirven?

Imagina que tienes una calculadora y quieres asegurarte de que suma correctamente. Podrías probarla manualmente cada vez que cambias algo, pero eso es lento y propenso a errores. Los tests son programas que comprueban automáticamente que el código funciona correctamente.

En este proyecto se usa **Pytest** — una librería de Python que ejecuta todos los tests automáticamente y te dice cuáles pasan y cuáles fallan. Se ejecuta con:

```
python -m pytest -v
```

`test_models.py` — Pruebas de las clases

```python
class TestModels:

    def test_creacion_juego_fisico(self):
        juego = JuegoFisico("Zelda", "NES", 1986, "Aventura", "Estante A1")
        assert juego.titulo == "Zelda"
        assert juego.plataforma == "NES"
        assert juego.anio == 1986
```

`assert` es la palabra clave de los tests — comprueba que una condición es verdadera. Si no lo es, el test falla y Pytest lo marca en rojo.

Este test crea un `JuegoFisico` y comprueba que sus atributos tienen los valores correctos. Parece obvio, pero es útil para detectar si alguien rompe el constructor accidentalmente al hacer cambios.

```python
    def test_validacion_anio(self):
        with pytest.raises(ValueError):
            JuegoFisico("Juego Futuro", "PC", 2099, "RPG", "Estante B1")
```

`pytest.raises(ValueError)` comprueba que el código dentro del bloque lanza exactamente ese tipo de error. Si no lo lanza — o lanza otro error — el test falla.

Este test verifica que la validación del año funciona correctamente — un año futuro debe lanzar `ValueError`.

```python
    def test_polimorfismo_mostrar_detalle(self):
        fisico = JuegoFisico("Mario", "NES", 1985, "Plataformas", "Estante C2")
        digital = JuegoDigital("Sonic", "PC", 1991, "Plataformas", "https://...")
        assert "[FÍSICO]" in fisico.mostrar_detalle()
        assert "[DIGITAL]" in digital.mostrar_detalle()
```

Este test verifica el polimorfismo — que cada subclase implementa `mostrar_detalle()` de forma diferente y correcta.

`test_database.py` — Pruebas de la base de datos

```python
class TestDatabase:

    @pytest.fixture
    def db(self):
        test_db = "test_db_logic.db"
        manager = DatabaseManager(test_db)
        yield manager
        manager.conn.close()
        if os.path.exists(test_db):
            os.remove(test_db)
```

`@pytest.fixture` es un decorador especial de Pytest. Una fixture es código que se ejecuta antes y después de cada test para preparar el entorno.

Esta fixture crea una base de datos temporal llamada `test_db_logic.db` antes de cada test y la elimina después. Así cada test empieza con una base de datos limpia y vacía — los tests no se afectan entre sí.

`yield manager` es el punto donde se ejecuta el test. Todo lo que hay antes del `yield` se ejecuta antes del test (preparación) y todo lo que hay después se ejecuta después (limpieza).

```python
    def test_insertar_y_listar(self, db):
        db.insertar_juego("Metroid", "NES", 1986, "Acción", "Físico", "E1")
        registros = db.obtener_coleccion_completa()

        assert len(registros) > 0
        assert registros[0][1] == "Metroid"
        assert registros[0][4] == "Acción"
```

Este test inserta un juego, lo recupera y comprueba que los datos son correctos.

`registros[0]` es la primera fila devuelta. `registros[0][1]` es la segunda columna de esa fila — el título. `registros[0][4]` es la quinta columna — el género.

```python
    def test_editar_juego(self, db):
        db.insertar_juego("Sonic", "Genesis", 1991, "Plataformas", "Físico", "E1")
        registros_pre = db.obtener_coleccion_completa()
        id_v = registros_pre[0][0]

        db.editar_juego(id_v, "Sonic 1", "Genesis", 1991, "Plataformas", "E2")
        registros_post = db.obtener_coleccion_completa()

        assert registros_post[0][1] == "Sonic 1"
        assert registros_post[0][6] == "E2"
```

Este test verifica que editar funciona correctamente — inserta un juego, lo edita y comprueba que los cambios se han guardado.

`registros_pre[0][0]` obtiene el ID del juego recién insertado — necesario para editarlo.

```python
    def test_relacion_prestamos(self, db):
        db.insertar_juego("Zelda", "NES", 1986, "Aventura", "Físico", "E1")
        registros = db.obtener_coleccion_completa()
        id_v = registros[0][0]

        db.registrar_prestamo(id_v, "Usuario Prueba")
        registros_con_prestamo = db.obtener_coleccion_completa()

        assert registros_con_prestamo[0][7] == "Usuario Prueba"
```

Este test verifica que el sistema de préstamos funciona — inserta un juego, lo presta y comprueba que el destinatario aparece en la consulta.

`registros_con_prestamo[0][7]` es la columna 7 — el destinatario del préstamo que viene del LEFT JOIN.

`test_analysis.py` — Pruebas del análisis

```python
class TestAnalysis:

    @pytest.fixture
    def setup_data(self):
        db_name = "test_analysis.db"
        db = DatabaseManager(db_name)
        db.insertar_juego("Juego A", "PC", 2000, "RPG", "Digital", "URL")
        yield db_name
        db.conn.close()
        if os.path.exists(db_name):
            os.remove(db_name)

    def test_datos_para_analisis(self, setup_data):
        conn = sqlite3.connect(setup_data)
        df = pd.read_sql_query("SELECT plataforma, anio, genero FROM juegos", conn)
        conn.close()

        assert not df.empty
        assert df["plataforma"].iloc[0] == "PC"
        assert "genero" in df.columns
```

`df.empty` devuelve `True` si el DataFrame no tiene filas. `not df.empty` comprueba que sí hay datos.

`df["plataforma"].iloc[0]` accede al primer valor de la columna plataforma. `.iloc[0]` significa "fila número 0" — es la forma correcta de acceder a un valor concreto en Pandas.

`"genero" in df.columns` comprueba que la columna `genero` existe en el DataFrame.

`test_robustez.py` — Pruebas de entradas incorrectas

```python
def test_error_anio_futuro():
    with pytest.raises(ValueError):
        JuegoDigital("Test", "PC", 2099, "RPG", "http://test.com")

def test_tipos_datos_incorrectos():
    with pytest.raises(Exception):
        JuegoFisico("Test", "NES", "no_es_un_año", "Acción", "Estante A")
```

Estos tests comprueban que el sistema falla correctamente ante entradas incorrectas. Un año futuro debe lanzar `ValueError`. Un año que no es un número debe lanzar algún tipo de excepción.

Probar que los errores funcionan correctamente es tan importante como probar que las cosas funcionan bien.

El resultado final

Cuando ejecutas `python -m pytest -v` ves esto:

```
tests/test_models.py::TestModels::test_creacion_juego_fisico      PASSED
tests/test_models.py::TestModels::test_validacion_anio            PASSED
tests/test_models.py::TestModels::test_polimorfismo_mostrar_detalle PASSED
tests/test_database.py::TestDatabase::test_insertar_y_listar      PASSED
tests/test_database.py::TestDatabase::test_editar_juego           PASSED
tests/test_database.py::TestDatabase::test_relacion_prestamos     PASSED
tests/test_analysis.py::TestAnalysis::test_datos_para_analisis    PASSED
tests/test_robustez.py::test_error_anio_futuro                    PASSED
tests/test_robustez.py::test_tipos_datos_incorrectos              PASSED

9 passed in 0.53s
```